

Experiment No. 5

Title:

Write a program to find the longest common subsequence between two sequences.

Aim:

The aim of this program is to find the longest common subsequence (LCS) between two given sequences. The LCS of two sequences is the longest sequence that can be derived from both original sequences by deleting some or no elements without changing the order of the remaining elements. This problem has applications in bioinformatics, text comparison, and various other fields.

Theory:

What is Dynamic Programming (DP)?

Dynamic Programming (DP) is a method used in mathematics and computer science to solve complex problems by breaking them down into simpler subproblems. By solving each subproblem only once and storing the results, it avoids redundant computations, leading to more efficient solutions for a wide range of problems.

How Does Dynamic Programming (DP) Work?

- **Identify Subproblems:** Divide the main problem into smaller, independent subproblems.
- **Store Solutions:** Solve each subproblem and store the solution in a table or array.
- **Build Up Solutions:** Use the stored solutions to build up the solution to the main problem.
- **Avoid Redundancy:** By storing solutions, DP ensures that each subproblem is solved only once, reducing computation time.

Examples of Dynamic Programming (DP)

Example 1: Consider the problem of finding the Fibonacci sequence:

Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

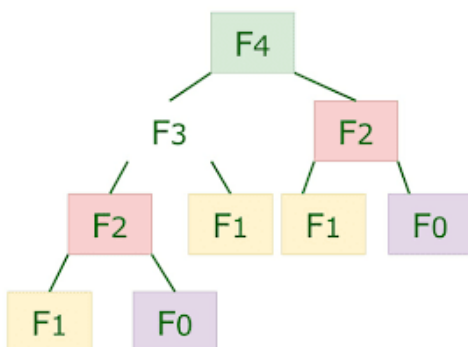
Brute Force Approach:

To find the n th Fibonacci number using a brute force approach, you would simply add the $(n-1)$ th and $(n-2)$ th Fibonacci numbers. This would work, but it would be inefficient for large values of n , as it would require calculating all the previous Fibonacci numbers.

Dynamic Programming Approach:

Nth Term of Fibonacci Series

Here F_n denotes Nth Term of the Fibonacci



Fibonacci Series using Dynamic Programming

- **Subproblems:** $F(0)$, $F(1)$, $F(2)$, $F(3)$, ...
- **Store Solutions:** Create a table to store the values of $F(n)$ as they are calculated.
- **Build Up Solutions:** For $F(n)$, look up $F(n-1)$ and $F(n-2)$ in the table and add them.
- **Avoid Redundancy:** The table ensures that each subproblem (e.g., $F(2)$) is solved only once.

By using DP, we can efficiently calculate the Fibonacci sequence without having to recompute subproblems.

Example 2: Longest common subsequence (finding the longest subsequence that is common to two strings)

Example 3: Shortest path in a graph (finding the shortest path between two nodes in a graph)

Example 4: Knapsack problem (finding the maximum value of items that can be placed in a knapsack with a given capacity)

When to Use Dynamic Programming (DP)?

Dynamic programming is an optimization technique used when solving problems that consists of the following characteristics:

1. Optimal Substructure:

Optimal substructure means that we combine the optimal results of subproblems to achieve the optimal result of the bigger problem.

Example:

*Consider the problem of finding the **minimum cost** path in a weighted graph from a **source** node to a **destination** node. We can break this problem down into smaller subproblems:*

- Find the **minimum cost** path from the **source** node to each **intermediate** node.
- Find the **minimum cost** path from each **intermediate** node to the **destination** node.

The solution to the larger problem (finding the minimum cost path from the source node to the destination node) can be constructed from the solutions to these smaller subproblems.

2. Overlapping Subproblems:

The same subproblems are solved repeatedly in different parts of the problem.

Example:

*Consider the problem of computing the **Fibonacci series**. To compute the Fibonacci number at index **n**, we need to compute the Fibonacci numbers at indices **n-1** and **n-2**. This means that the subproblem of computing the Fibonacci number at index **n-1** is used twice in the solution to the larger problem of computing the Fibonacci number at index **n**.*

Approaches of Dynamic Programming (DP)

Dynamic programming can be achieved using two approaches:

1. Top-Down Approach (Memoization):

In the top-down approach, also known as **memoization**, we start with the final solution and recursively break it down into smaller subproblems. To avoid redundant calculations, we store the results of solved subproblems in a memoization table.

Let's breakdown Top down approach:

- Starts with the final solution and recursively breaks it down into smaller subproblems.
- Stores the solutions to subproblems in a table to avoid redundant calculations.
- Suitable when the number of subproblems is large and many of them are reused.

2. Bottom-Up Approach (Tabulation):

In the bottom-up approach, also known as **tabulation**, we start with the smallest subproblems and gradually build up to the final solution. We store the results of solved subproblems in a table to avoid redundant calculations.

Let's breakdown Bottom-up approach:

- Starts with the smallest subproblems and gradually builds up to the final solution.
- Fills a table with solutions to subproblems in a bottom-up manner.
- Suitable when the number of subproblems is small and the optimal solution can be directly computed from the solutions to smaller subproblems.

Dynamic Programming (DP) Algorithm

Dynamic programming is an algorithmic technique that solves complex problems by breaking them down into smaller subproblems and storing their solutions for future use. It is particularly effective for problems that contain **overlapping subproblems** and **optimal substructure**.

Common Algorithms that Use Dynamic Programming:

- **Longest Common Subsequence (LCS):** Finds the longest common subsequence between two strings.
- **Shortest Path in a Graph:** Finds the shortest path between two nodes in a graph.
- **Knapsack Problem:** Determines the maximum value of items that can be placed in a knapsack with a given capacity.
- **Matrix Chain Multiplication:** Optimizes the order of matrix multiplication to minimize the number of operations.
- **Fibonacci Sequence:** Calculates the nth Fibonacci number.

Advantages of Dynamic Programming (DP)

Dynamic programming has a wide range of advantages, including:

- Avoids recomputing the same subproblems multiple times, leading to significant time savings.
- Ensures that the optimal solution is found by considering all possible combinations.
- Breaks down complex problems into smaller, more manageable subproblems.

Applications of Dynamic Programming (DP)

Dynamic programming has a wide range of applications, including:

- **Optimization:** Knapsack problem, shortest path problem, maximum subarray problem
- **Computer Science:** Longest common subsequence, edit distance, string matching
- **Operations Research:** Inventory management, scheduling, resource allocation

Longest Common Subsequence (LCS)

The Longest Common Subsequence (LCS) problem is a classic problem in computer science, particularly in the field of dynamic programming. Given two sequences, the goal is to find the longest subsequence

present in both sequences. A subsequence is a sequence derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Applications

- **Bioinformatics:** Comparing DNA, RNA, or protein sequences.
- **Text Comparison:** Differencing files, version control systems.
- **Data Compression:** Identifying repeating patterns.

Examples:

Input: S1 = "ABC", S2 = "ACD"

Output: 2

Explanation: The longest subsequence which is present in both strings is "AC".

Input: S1 = "AGGTAB", S2 = "GXTXAYB"

Output: 4

Explanation: The longest common subsequence is "GTAB".

Input: S1 = "ABC", S2 = "CBA"

Output: 1

Explanation: There are three common subsequences of length 1, "A", "B" and "C" and no common subsequence of length more than 1.

Input: S1 = "XYZW", S2 = "XYWZ"

Output: 3

Explanation: There are two common subsequences of length 3 "XYZ", and "XYW", and no common subsequence of length more than 3.

Pseudocode:

FUNCTION LCS(X, Y)

 m <- LENGTH(X)

 n <- LENGTH(Y)

 lcs_table <- ARRAY[m + 1][n + 1] of INTEGER

 FOR i FROM 0 TO m

 FOR j FROM 0 TO n

 IF i == 0 OR j == 0 THEN

 lcs_table[i][j] <- 0

 ELSE IF X[i - 1] == Y[j - 1] THEN

 lcs_table[i][j] <- lcs_table[i - 1][j - 1] + 1

 ELSE

 lcs_table[i][j] <- MAX(lcs_table[i - 1][j], lcs_table[i][j - 1])

```

        END IF
    END FOR
END FOR

index <- lcs_table[m][n]
lcs_string <- ARRAY[index + 1] of CHARACTER
lcs_string[index] <- '\0'
i <- m
j <- n
WHILE i > 0 AND j > 0 DO
    IF X[i - 1] == Y[j - 1] THEN
        lcs_string[index - 1] <- X[i - 1]
        i <- i - 1
        j <- j - 1
        index <- index - 1
    ELSE IF lcs_table[i - 1][j] > lcs_table[i][j - 1] THEN
        i <- i - 1
    ELSE
        j <- j - 1
    END IF
END WHILE
RETURN JOIN (lcs_string)
END FUNCTION

# Example usage
X <- "AGGTAB"
Y <- "GXTXAYB"
PRINT "Longest Common Subsequence: ", LCS(X, Y)

```

Experiment 5 Title:

Experiment 5 Code:

Experiment 5 Output (Screenshot):**Bucket List Experiment Title:****Bucket List Experiment Code:****Bucket List Experiment Output (Screenshot):****Theory Questions (Handwritten):**

Q1: Compare and contrast the dynamic programming approaches used to solve the Fibonacci sequence, the longest common subsequence (LCS), and the knapsack problem. How do the bottom-up and top-down approaches apply to these problems?

Q2: Explain the bottom-up and top-down approaches in dynamic programming. How do these approaches differ in terms of implementation and efficiency when applied to problems such as the Fibonacci sequence and the knapsack problem?

Conclusion:

Hence, we studied to determine the longest common subsequence between two sequences by employing a dynamic programming approach, which efficiently computes the optimal subsequence length through iterative table-filling.